



Fundamentos da Arquitetura de Sistemas

Você vai compreender a importância das metodologias na condução de projetos e conhecer a UML como ferramenta para modelagem e desenvolvimento de sistemas baseados em componentes.

Prof. Marcos Alexandre Fernandes Ferronato

Propósito

O domínio de metodologias de desenvolvimento e modelagem, como a UML, é fundamental para profissionais da área de tecnologia elaborarem sistemas mais organizados, eficazes e alinhados às necessidades do negócio, favorecendo a atuação estratégica em projetos de arquitetura de sistemas.

Objetivos

- Distinguir os elementos que caracterizam um componente e sua arquitetura, reconhecendo sua função, especificação, implementação e diferenças em relação à arquitetura de sistemas.
- Relacionar os processos de desenvolvimento e de gestão dentro da arquitetura de sistemas, avaliando sua integração e importância para o sucesso dos projetos.
- Explicar o papel dos modelos UML no desenvolvimento de software por componentes, destacando sua contribuição para a estruturação e confiabilidade da arquitetura de sistemas.

Introdução

Gerenciar a complexidade é um dos principais desafios no desenvolvimento de sistemas. Uma das formas mais eficazes de lidar com esse desafio é adotar o princípio de “dividir para conquistar” — ou seja, quebrar um problema grande em partes menores, mais simples de compreender e de implementar. Essa é a lógica por trás do uso de componentes em sistemas de informação: dividir um sistema em módulos menores, como objetos de negócio, dados e interfaces, que são desenvolvidos separadamente e integrados posteriormente para formar uma solução robusta e funcional.

Para que essa abordagem funcione com eficiência, é essencial utilizar metodologias que orientem todo o processo, desde a identificação dos componentes até sua implementação e integração. Metodologias são conjuntos organizados de processos, boas práticas e diretrizes que auxiliam a equipe no planejamento, desenvolvimento e entrega de sistemas de forma mais segura e estruturada — mesmo quando os profissionais ainda estão em fase de aprendizagem ou possuem pouca experiência.

Neste conteúdo, exploraremos a importância de aplicar metodologias tanto de gestão quanto de desenvolvimento no contexto da arquitetura de sistemas. Veremos como essas abordagens se complementam e como o uso de modelos conceituais, técnicos e visuais favorece a construção de sistemas eficazes e eficientes. Em especial, destacaremos o papel da Unified Modeling Language (UML), uma linguagem de modelagem amplamente utilizada para representar visualmente a estrutura e o comportamento dos sistemas. Por meio da UML, é possível documentar e comunicar as decisões do projeto de forma clara, contribuindo para o alinhamento entre equipes e para o sucesso das soluções desenvolvidas.

Fundamentos de componentes

Componentes, no contexto da arquitetura de sistemas, são **unidades de software estruturados** de acordo com alguns princípios específicos.

Os princípios fundamentais que regem os componentes estão relacionados ao conceito de objetos, conforme descrito a seguir:

União de dados e funções

Um objeto de software consiste em dados que podem assumir valores e funções que tratam esses dados. Os **dados e funções** devem ter uma ligação natural entre eles, formando o conceito de classe.



Encapsulamento

Conceito de esconder de quem vai usar a classe os detalhes de sua funcionalidade e de dados, deixando amostra somente como **acionar e o resultado a ser alcançado** pelo acionamento.

Não importa para quem vai usar tal componente, como as coisas acontecem dentro dele, e, sim, se o resultado esperado foi alcançado.



Identidade

Cada componente encapsulado tem uma **identidade única de dados e funções** e pode assumir estados pré-determinados.

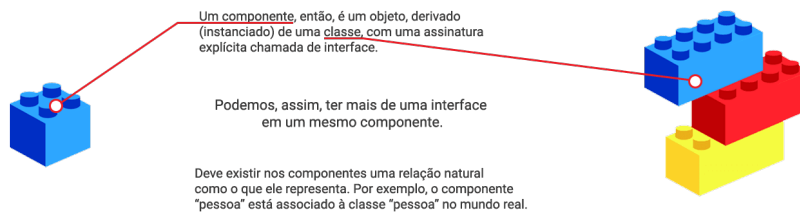


O que é um componente afinal?

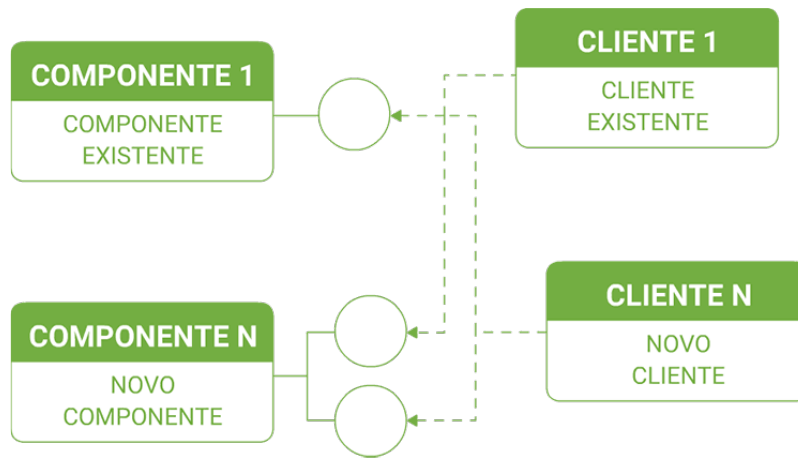


Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Arquitetura de sistemas baseada em componentes



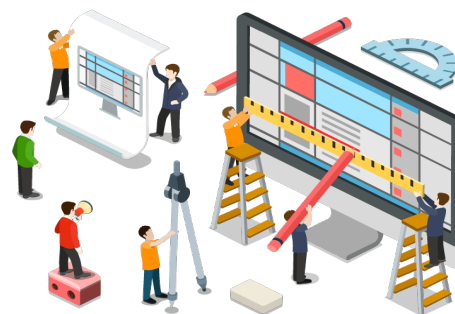
A arquitetura de sistemas baseada em componentes, então, está relacionada à identificação das interfaces possíveis e disponíveis e como elas resolvem os problemas que se apresentam.

Esse conceito facilita muito o desenvolvimento e reduz o nível de mudança nos sistemas gerados.

Objetivos de componentes

A partir de um objeto simples estruturado (componente), é necessário combinar funcionalidade e dados para resolver os problemas em sistemas computacionais.

O grande desafio na abordagem por componentes na arquitetura de sistemas está em conseguirmos identificar claramente as interfaces e como elas resolvem os problemas que se apresentam. Isso está relacionado às dependências entre componentes.



Pode até ser uma surpresa para alguns que, quando definem seus objetos, estão pensando somente na reutilização e não em como suas interfaces se relacionam em contextos variados.



Atenção

Quando pensamos em componentes a partir de suas interfaces, criamos um número maior de possibilidades de uso e reduzimos significativamente o número de mudanças futuras.

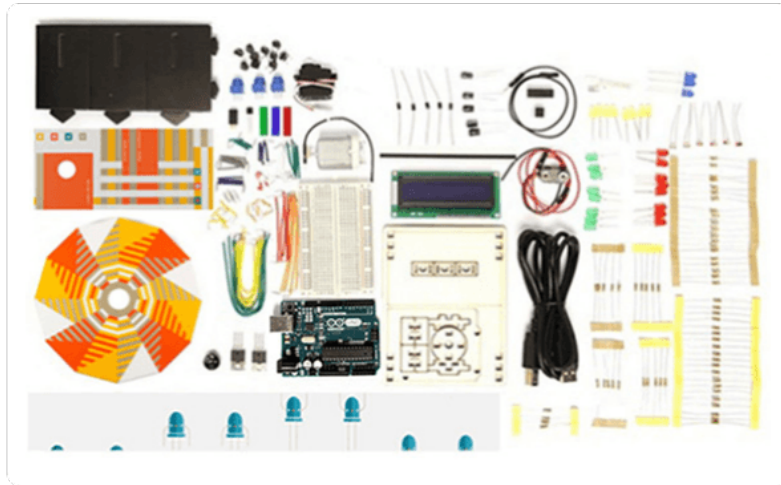
Características de componentes

Existem vários conceitos que podem definir os objetivos de um componente, todos com real validade, mas podemos afirmar que um componente contém as características necessárias para resolver problemas de forma natural e relacionado ao mundo real.

Vamos ver algumas delas:

- Ser capaz de montar uma aplicação, ligando componentes que devem atender a padrões pré-definidos e estejam agrupados por categoria, dentro do conceito de "caixa de componentes". Esses componentes devem se encaixar a outros (através de suas interfaces), para resolver os problemas.
- O uso dado aos componentes, então, pode variar, de acordo com a necessidade e experiência de cada arquiteto de sistemas.
- Quando você, arquiteto de sistemas, for utilizar os componentes para criar suas aplicações, é necessário identificar bem as interfaces e como estas se ligam a outros componentes.
- Para que isso seja possível é necessário que as interfaces estejam bem especificadas e claras.
- Se as interfaces dos componentes estiverem bem definidas, será possível trocar componentes sem afetar os resultados da aplicação e sem a necessidade de mudanças de códigos.
- Caso não estejam bem definidas, essas facilidades e funcionalidades podem ser perdidas.
- As características de funcionalidade de um determinado componente devem estar claras na definição das interfaces (Por exemplo, um componente que atualiza endereço através de CEP, somente poderá ser utilizado em uma aplicação que queira funcionar nos EUA se também tratar ZIP CODE, que é o equivalente ao CEP para este país).

Sobre o segundo item, como mostra a imagem para sistemas de automação, componentes eletrônicos têm as mesmas características de componentes de sistemas:



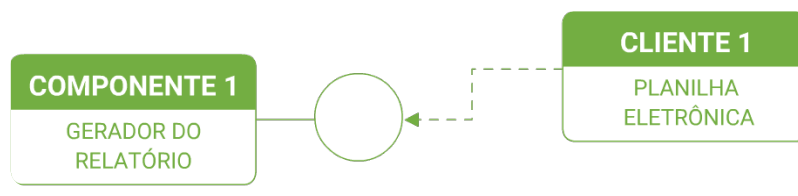
Exemplo de uso de componentes

Imagine que nosso sistema deve gerar um relatório em uma planilha eletrônica para que o usuário tenha como trabalhar com as informações e montar seus relatórios e gráficos personalizados.

É bem razoável que não esteja no escopo do desenvolvimento do sistema a construção de um sistema de planilha eletrônica. É muito mais simples e eficaz utilizarmos o Excel ou a Planilha do OpenOffice para este fim.

Para que isso ocorra, precisamos mapear as interfaces de uso dessas planilhas e que elas sejam repassadas aos desenvolvedores dos novos componentes de maneira que se liguem ao componente “Planilha Eletrônica que se deseja utilizar”.

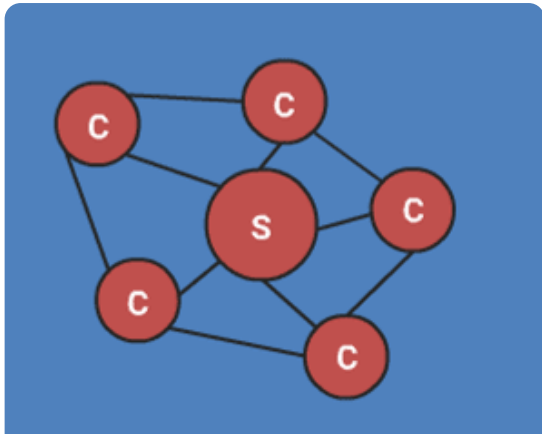
Graficamente, teríamos o seguinte:



O que não é um componente? O fato de termos uma classe ou uma função escrita em uma linguagem de programação, compilada e instalada em nosso ambiente, por si só não atribui a esse código a funcionalidade de um componente de software.

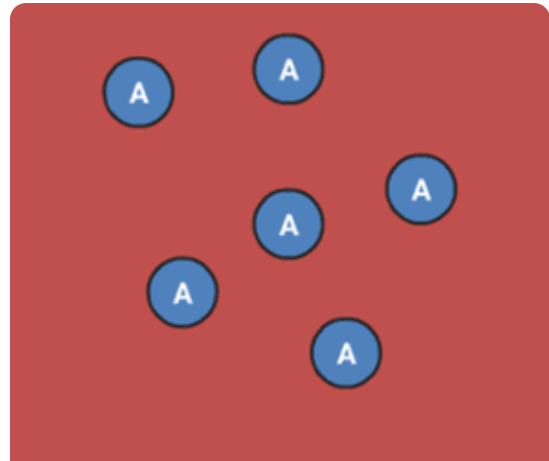
É necessário que tenhamos uma interface bem definida e que padrões de interação sejam suportados pelo componente. Somente neste caso ele estará apto a ser utilizado e classificado como componente.

Definição de arquitetura de sistemas e componentes



Arquitetura de sistema

Usaremos a seguinte definição para arquitetura de sistema: é um conjunto de componentes que compõem um software completo instalado na corporação, incluindo as funcionalidades destes componentes, a sua interconexão e, possivelmente, até mesmo a tecnologia adequada.



Arquitetura de componentes

Já para arquitetura de componentes usaremos o seguinte: é um conjunto de componentes de software em nível de aplicativo, com sua situação estrutural, relacionamentos e suas dependências comportamentais claramente definidas e liberadas para uso.

Arquiteturas de sistemas

Podemos ter **N** camadas de arquiteturas distribuídas, ligando bancos de dados corporativos, pacotes de automação e sistemas em funcionamento na corporação, interagindo especificamente através dos aplicativos de processos de negócios software com as interfaces de usuário baseadas na web.





Atenção

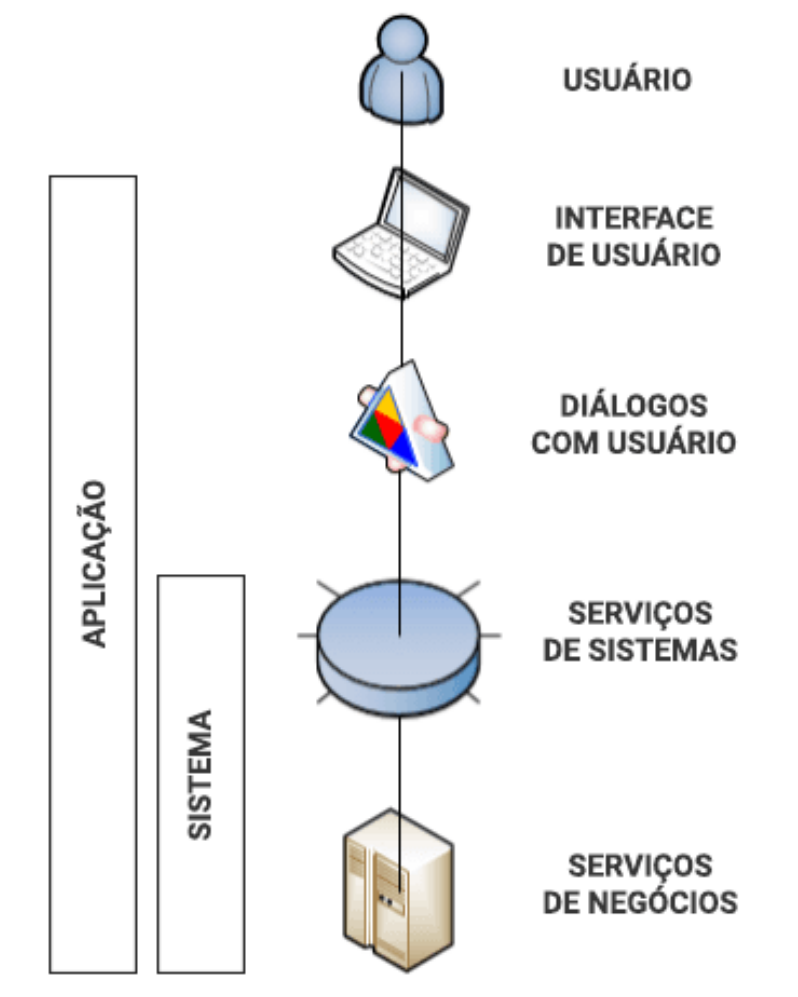
Essa é uma típica arquitetura do sistema para os tipos de sistemas que podemos explorar nesta disciplina. Compreender esta questão relacionada à arquitetura do sistema é importante porque nos diz a forma geral do sistema final e explica como usaremos várias tecnologias para montar o sistema que precisamos.

Camadas da arquitetura de sistemas

Queremos usar componentes para fins diferentes e para resolver preocupações diferentes.

Nossa abordagem global é identificar as camadas diferentes nas quais os componentes podem ser utilizados.

Isso é útil porque nos permite raciocinar sobre a finalidade de cada unidade de software que usaremos em nosso sistema.



Arquitetura de componentes

Componentes podem ser encontrados em qualquer uma das camadas da Arquitetura de sistemas, conforme imagem anterior. Porém, o que nos interessa aqui são os componentes presentes nas camadas de sistemas e de negócio.

Como vimos, a arquitetura de componentes é um conjunto elementos de software no nível de aplicativo que contém comportamentos e dependências em suas relações estruturais.

Essa é uma definição independente do nível de tecnologia em que será implantado. A arquitetura de componentes pode ser usada em uma única aplicação ou para um contexto mais amplo, como um conjunto de aplicações que servem uma área de processo de negócio em particular.

Dentro desse contexto, os componentes nos permitem entender como, dependendo do nível de integração (forte, moderada ou fraca), nosso sistema vai reagir às modificações e/ou à substituição de componentes.

Relação entre especificação de componentes e interfaces

A especificação componente define a assinatura do componente e, conseqüentemente, a forma como será construído, utilizado e testado.

Essa especificação define as formas de uso e delimita sua fronteira de acesso. Já a interface define a relação com os outros componentes, informando o que esperar quando se conectar a este componente.



Níveis de modelo

Um modelo de algo apresenta uma perspectiva ou visão resumida do que é e do que aquilo permite. Deve ficar claro que sempre em um modelo algumas coisas serão enfatizadas e outras serão excluídas.

Categorias de modelos

Modelo de implementação

Informa os detalhes de implementação que devem estar presentes dentro dos códigos.

Modelo de especificação

Representa os componentes de software a serem utilizados.

Modelo conceitual

Independentemente do tipo de software ou de tecnologia, representa o problema a ser resolvido.

É importante frisar também que existem modelos conceituais mais fortes e modelos conceituais mais fracos.

Workflow

Workflow representa a metodologia de desenvolvimento de sistemas baseada na metodologia RUP.

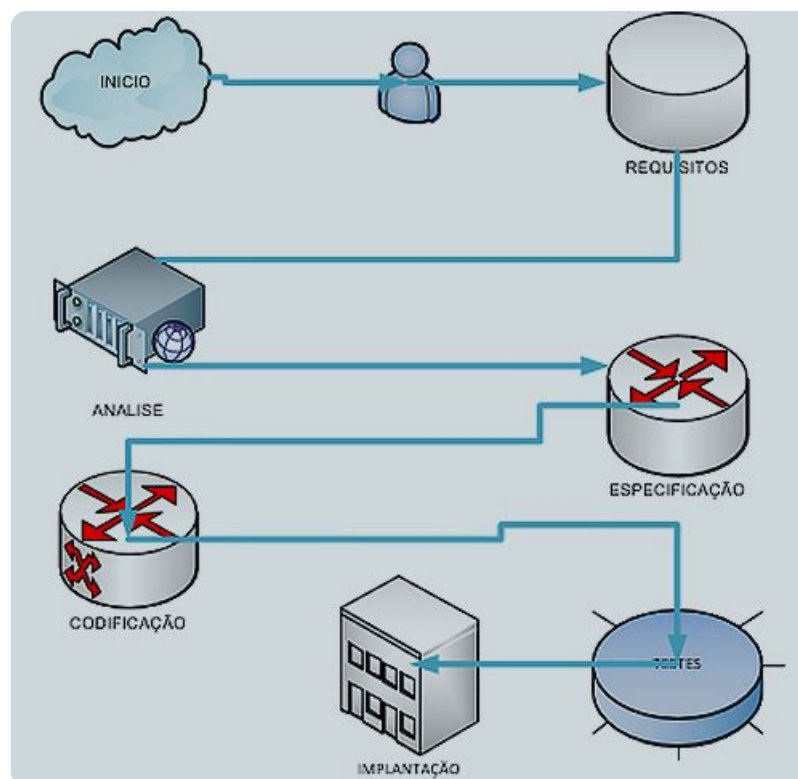
Parte-se da ideia do sistema, em seguida efetua-se a coleta de requisitos que, após validados, são encaminhados para a análise.

Depois que os modelos conceituais de negócio são desenvolvidos, o processo é encaminhado para a especificação das funcionalidades e interfaces de sistemas.

Ao final desse processo, as especificações são encaminhadas para codificação pela equipe de desenvolvimento, para, em seguida, serem efetuados os testes integrados e por último a implantação do novo sistema.

Metodologia

Ramo da lógica que se ocupa dos métodos, do como fazer.



O gerenciamento de processos

Refere-se ao conjunto de conhecimentos que serão utilizados para guiar a condução do projeto de desenvolvimento de software.

Esses conjuntos de processos garantem que o resultado dos projetos será um sucesso, baseado nos grupos

de processos que contém ações, distribuídas em 10 áreas do conhecimento da Gestão de Projetos, segundo o PMI®:

- Iniciação;
- Planejamento;
- Execução;
- Monitoramento e Controle;
- Encerramento da Gestão.

Cada um dos grupos de processos se integra às áreas de conhecimento em uma sequência lógica que, quando seguida, viabiliza a execução do desenvolvimento com muito mais assertividade.

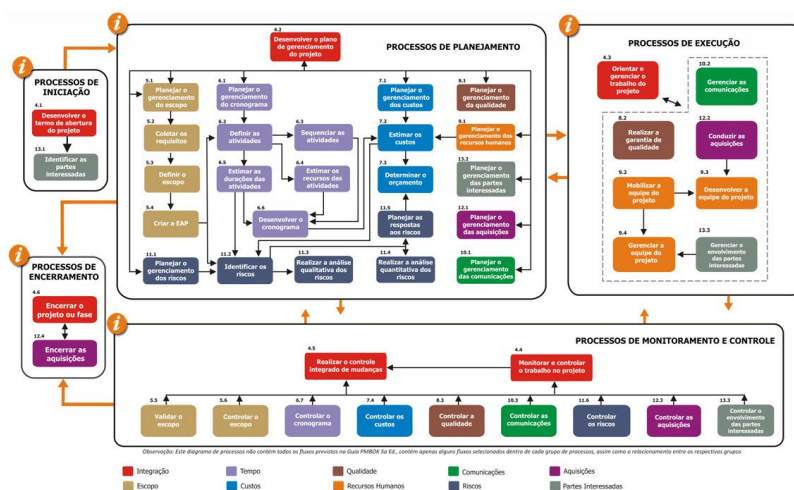
Gestão

Ato ou efeito de gerir ou administrar um processo produtivo.



Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Diferenças entre métodos

Em alguns métodos de desenvolvimento, prega-se que os requisitos devam ser levantados até ser alcançada sua totalidade, independentemente do tempo que isso demore, com o objetivo de reduzir mudanças futuras.

Outros métodos pregam que os requisitos mínimos devam ser levantados para iniciar ondas de desenvolvimento, e, com o passar do tempo, chegam a situação ideal sem perder muito tempo com requisitos. A justificativa seria: já que os requisitos vão mudar mesmo, então não se deve perder tempo com isso.

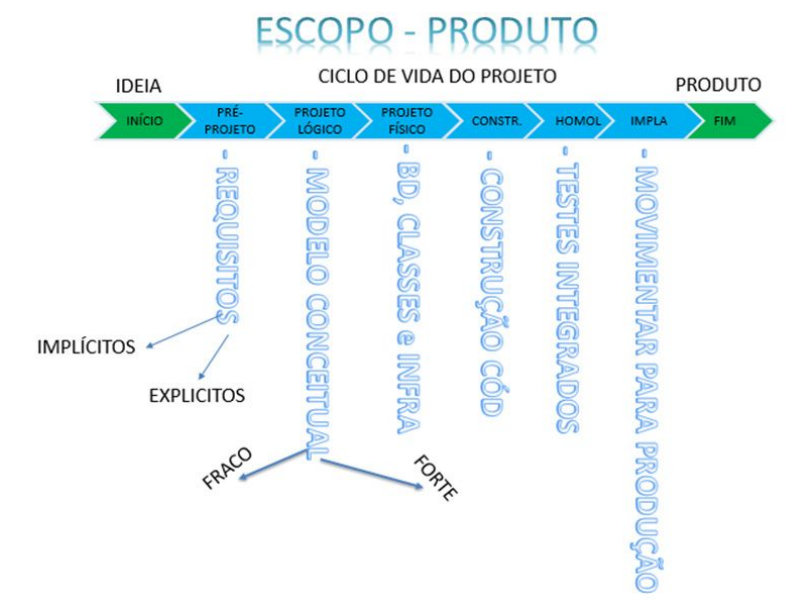


Atenção

Faremos aqui uma abordagem intermediária, onde se gasta no máximo 5% do esforço total do projeto com levantamento de requisitos e, ao mesmo tempo, garante-se que eles tenham sido bem definidos e validados. Isso somente é possível com a prototipação para validar requisitos em tempo de modelagem conceitual.

Fica claro que vão existir trabalhos de gestão e trabalhos de desenvolvimento.

Os dois tipos de trabalhos vão coexistir em workflows diferentes, que interagem e se integram para gerar um sistema muito mais assertivo do ponto de vista de funcionalidade e em conformidade com escopo, tempo, custo, riscos e qualidade.



Objetivos da metodologia de desenvolvimento

A metodologia de desenvolvimento tem como objetivo guiar o processo de produção de software, de forma que os componentes gerados tenham alta qualidade e sejam produzidos mais rapidamente, garantindo sua efetividade.

Lembre-se que é objetivo de uma metodologia definir, de forma clara:

- Quem?

- O que?
- Quando?
- Como?
- Onde?

Outro ponto importante em uma metodologia é o conjunto de padrões a serem seguidos para garantir o uso de boas práticas e que as funcionalidades sejam construídas conforme seus requisitos.

Objetivos da metodologia de gestão

A metodologia de gestão deve contemplar quantas fases forem necessárias para conseguir que todas as áreas de conhecimento sejam abordadas de forma a garantir que escopo, tempo, custos e qualidade atinjam os níveis definidos pelas corporações como sendo os ideais.

As fases da metodologia devem seguir um **modelo** interativo e incremental. Nele, cada fase é dividida em uma ou mais iterações que visam uma entrega ao final.

Veja a seguir um exemplo das fases de um processo de desenvolvimento, testes e manutenções dos software.





Atenção

Cada entrega de cada fase deve garantir que o resultado esteja com o grau de maturidade necessário naquele momento do projeto.

Interação entre componentes

A interação de componentes define como cada uma das operações do sistema será alcançada, utilizando a arquitetura de componentes.

Usa-se a interação entre os modelos para descobrir as operações nas interfaces de negócios. Quanto mais interações são consideradas, operações e padrões de uso comuns são conseguidos e passam a poder ser reutilizados.

Dessa maneira, as escolhas e possibilidades se tornam mais claras e as operações são movidas de uma interface para outra, quando necessário.

Grupos alternativos de interface para os componentes podem ser utilizados e este é o momento de pensar as integrações referenciais entre componentes para que os problemas sejam minimizados e as integridade sejam respeitadas. Assim, a interação de componentes é o momento em que todos os fatores do sistema são levantados, com uma clara compreensão das dependências entre eles, chegando-se até o nível mais detalhado de operações.



Especificação de componentes

É na fase final da especificação que ocorre o **detalhamento das operações** e as suas restrições.

Para uma dada interface, deve-se definir os potenciais estados dos componentes e suas assinaturas e, em seguida, especificar as condições prévias e posteriores para as operações. Aqui são levantadas ainda as regras de negócios e restrições.

As condições prévias e posteriores e outras restrições fazem referência aos tipos das informações de modelo de interface que, em conjunto com os tipos dos parâmetros, formam a assinatura da interface.



Saiba mais

Além desses detalhes na especificação da interface, esta etapa também apresenta a especificação de restrições, que são específicas para um componente e determinam como as definições de tipo de interfaces individuais vão corresponder a cada elemento no contexto desse componente.

A arquitetura não deve ser efetivamente alterada nesta fase. Essa tarefa de especificação detalhada somente deve ser realizada quando a definição da arquitetura estiver estável e todas as operações das interfaces forem identificadas.

O ato de escrever as regras detalhadas para cada operação pode ajudar você a descobrir parâmetros que estejam faltando ou informações que precisem ser complementadas, mas a ênfase está em identificar cada detalhe em uma arquitetura estável.

Verificando o aprendizado

Questão 1

Quais são as três características de um projeto de desenvolvimento de software?

- ☐ A Contínuo, Mensurável e Realizável.
- ☐ B Temporário, Mensurável e Realizável.
- ☒ C Temporário, Gera um resultado único e Elaborado Progressivamente.
- ☐ D Gera um resultado único, Mensurável e Realizável.



A alternativa C está correta.

Projetos de software são temporários, visam um resultado único e evoluem de forma progressiva.

Questão 2

Qual das respostas abaixo melhor define o conceito de ciclo de vida de projeto de desenvolvimento de software?

A As partes do projeto.

B Iniciação, planejamento, execução, monitoramento e controle e encerramento.

C As etapas que compõe o desenvolvimento de um projeto.

D As operações de um projeto.



A alternativa C está correta.

O ciclo de vida define as etapas que estruturam o desenvolvimento do projeto.

Questão 3

O processo de decomposição para definição do escopo de um projeto de desenvolvimento de software é uma técnica utilizada para construir um(a):

A Diagrama de rede de precedência

B Método de diagrama de caminho crítico.

C Estrutura Analítica do Projeto (EAP).

D Análise de variação.



A alternativa C está correta.

A decomposição do escopo resulta na Estrutura Analítica do Projeto (EAP).

Porque precisamos de UML na arquitetura de sistemas?

UML foi concebida para modelagem de projeto de sistemas orientado a objetos.

Esse conceito tem uma relação direta com o desenvolvimento orientado a componentes, do conceito de arquitetura de sistemas.



Atenção

Mas lembre-se que o foco desta disciplina está nos aspectos externos de componentes de sistemas e não no desenvolvimento dos módulos internos, mesmo que eles também possam ser modelados utilizando UML.

Neste contexto, um dos itens que usaremos em nossa disciplina é a modelagem de pacotes de interface, em que os componentes são modelados sem a preocupação com a codificação, mas, sim, com a junção dos elementos do sistema para definir sua lógica de funcionamento.

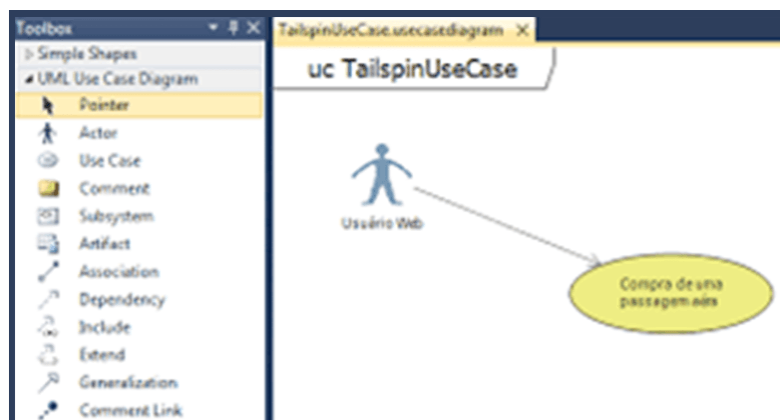
Veremos a seguir que UML pode nos ajudar a definir e projetar a arquitetura do sistema, servindo como ferramenta de suporte à técnica de modelagem por componentes da arquitetura de sistemas.

Não vamos aqui definir ou indicar uma ferramenta de modelagem UML específica, já que todas as ferramentas que se propõe a isso atendem aos objetivos propostos nesta disciplina.



Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.

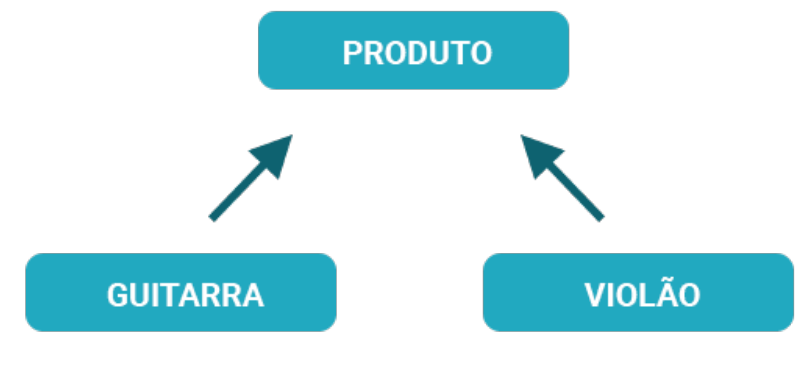


UML e suas generalizações

Na UML, temos um grande número de mecanismos de derivação de classes. Porém, com certeza, o mais utilizado na criação e implementação dos componentes é a generalização.

Qualquer classe UML pode sofrer generalização para implementar novas funcionalidades e/ou novos componentes.

Em um conceito mais amplo, sempre que você generalizar uma classe utilizando UML, pode também criar restrições específicas para essa generalização, independente da classe da qual este novo conceito foi estendido.



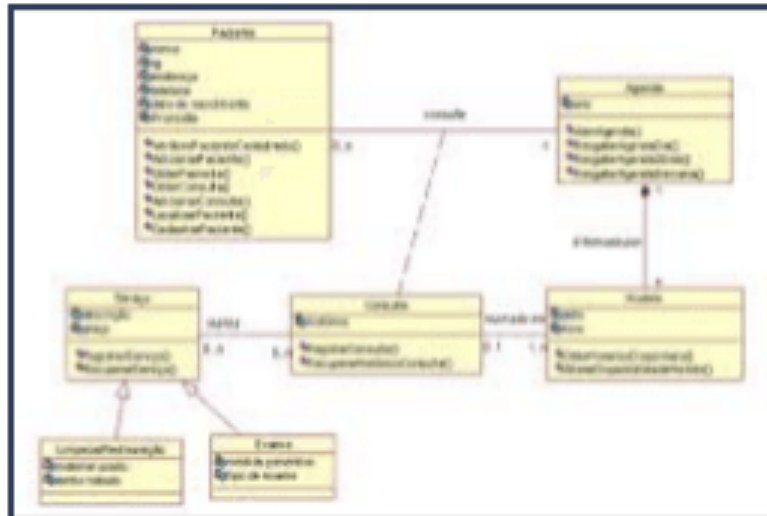
Como atingir conformidade em requisitos com UML?

1. Ao definirmos nossos componentes, é necessário que sejam geradas as suas especificações de comportamento.
2. É através desses componentes que vamos garantir que as funcionalidades levantadas estejam implementadas e em conformidade com os requisitos validados pelas partes interessadas.
3. Ao definirmos o comportamento deles, estamos sendo mais precisos, em relação à integração do modelo, e completos, em relação à sua funcionalidade.
4. Ao efetuarmos os testes de comportamento, durante a homologação, estamos garantindo que a aplicação esteja em conformidade com os requisitos.

Técnicas de modelagem UML

Veremos aqui como usar UML para modelar os vários artefatos necessários na modelagem de sistemas por componentes.

Esses artefatos nos servirão neste contexto para modelar o negócio e o sistema, na forma dos componentes que formam a arquitetura do sistema.



Técnicas de modelagem UML

Os diagramas que vamos utilizar usam os mesmos nomes da UML, mas são aqui apresentados com uma funcionalidade específica.

Exemplo

Para modelarmos os conceitos de negócios, usaremos o diagrama de classes, pois ele descreve o modelo de conceito do negócio.

Outro exemplo é o diagrama de especificação de interface que apresenta a especificação das interfaces. Nesse mesmo caminho temos os seguintes modelos:

- diagrama de modelo de negócio.
- diagrama de componentes e diagrama de implantação.

Temos também as seguintes variações:

Diagrama de funcionalidades de interface

Descreve o modelo de negócio, suas interfaces e as regras de funcionalidades para essas interfaces.

Diagrama de interação de componentes

É um diagrama de colaboração utilizado para interação entre componentes.

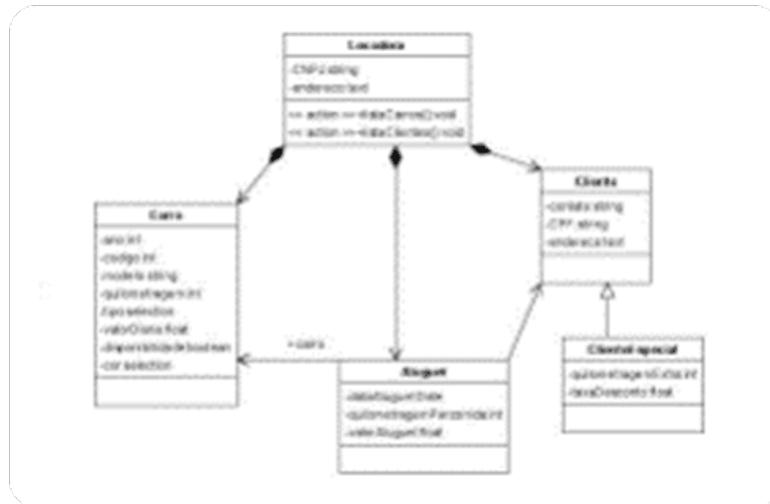
Veremos mais detalhadamente cada um deles.

Modelo conceitual de negócio

O modelo conceitual de negócio não é um modelo de software, mas sim um modelo de informação que define o domínio do problema.

Sua principal finalidade é identificar e definir as relações existentes dentro do negócio.

Usaremos o diagrama de classe UML para representá-lo, mas é importante ficar claro que a ferramenta é independente do modelo, ou seja, poderemos generalizar cada classe do modelo e amarrá-las do ponto de vista de restrições. Depois, quando formos definir os pacotes, poderemos separar o negócio do sistema.



Os modelos conceituais de negócio, normalmente, definem classes conceituais e suas associações e, como previsto nesse modelo, as associações terão suas multiplicidades.

O modelo pode conter atributos, caso isso seja significativo para explicar o negócio, mas seu uso não é obrigatório. A ênfase desse modelo é definir o domínio e não normalizar suas relações.

Modelo conceitual de sistemas

Diagrama de casos de uso

O diagrama de casos de uso é modelo que define a funcionalidade do sistema a ser desenvolvido e contém os seguintes elementos:

Atores

Representados por um boneco e um rótulo com o nome do ator.

Um ator é um usuário do sistema, que pode ser um usuário humano ou um outro sistema computacional.



Objetos

São representados por uma elipse e um rótulo com o nome do objeto.

Um objeto define uma grande função do sistema. A implicação é que uma função pode ser estruturada em outras funções e, portanto, um objeto pode ser explodido em níveis mais baixos ou detalhados.

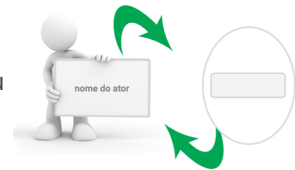


Relacionamentos

Podem ser associações entre atores e objetos, generalização entre atores ou generalizações (extends e includes) entre os objetos.

Os relacionamentos ajudam a representar o negócio dentro do diagrama de casos de uso.

O primeiro tipo de relacionamento que veremos é a associação, que representa a que objeto(s) os atores se relacionam ou, ainda, como um segundo ator se relaciona com os mesmos objetos do primeiro, a que está relacionado.



Relação entre objetos

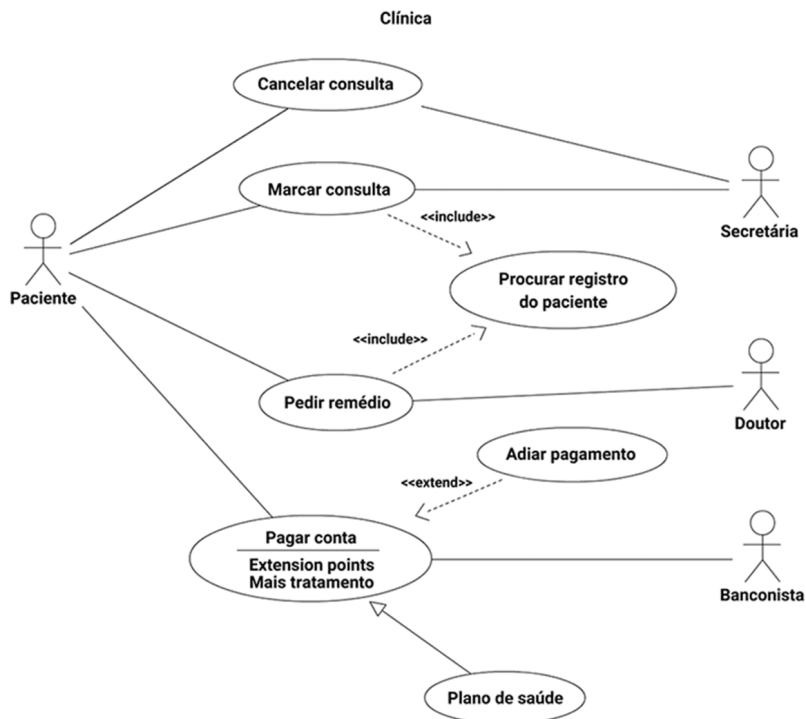
Quando estamos falando da relação entre objetos, temos o **include**, que informa que o objeto A é parte do objeto B, caso exista entre eles uma relação do tipo include partindo de A.

Outro tipo de relação entre objetos é o **extends**. Esse tipo de relação representa que o objeto D é uma extensão do objeto A, compartilhando suas funcionalidades e implementando algumas novas, que existem somente para ele.



Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.



Descrição de casos de uso

Descrevem, geralmente na forma textual, as funcionalidades existentes no diagrama de casos de uso, no que diz respeito a como seus atores interagem com os objetos do sistema.

Descrevem também como os objetos se relacionam entre si e, por último, a funcionalidade dos objetos.



Conteúdo interativo

Acesse a versão digital para ver mais detalhes da imagem abaixo.

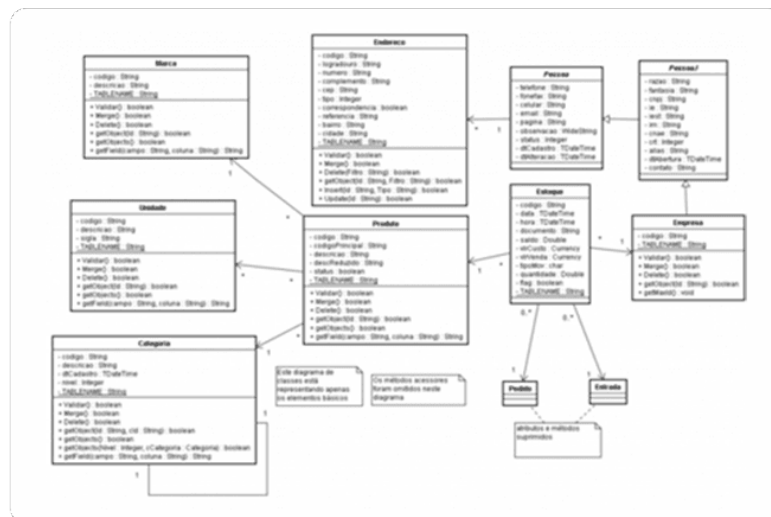
Nome do Caso de Uso	Manutenção de Cliente
Atores	Usuários, Administrador
Dados Consumidos	Nome, Endereço, Telefone, Email, CPF
Dados Produzidos	Código do Cliente
Prioridade	Alta
Fluxo Principal	
Ação do Ator	Resposta do Sistema
O usuário, após autenticar-se, poderá dar manutenção (incluir, alterar, excluir, consultar) nos clientes	
Incluir, alterar, excluir ou consultar	Apresentar formulário padrão
Fluxo Alternativo	
Ação do Ator	Resposta do Sistema

Diagrama de domínio

O diagrama de domínio é uma variação do diagrama de classes e utiliza quase a mesma notação.

Representa o domínio de dados a serem tratados e armazenados pelo sistema, ou seja, os elementos DAO do sistema.

Garante uma visão sistêmica entre os componentes de interface e os componentes dados do sistema.



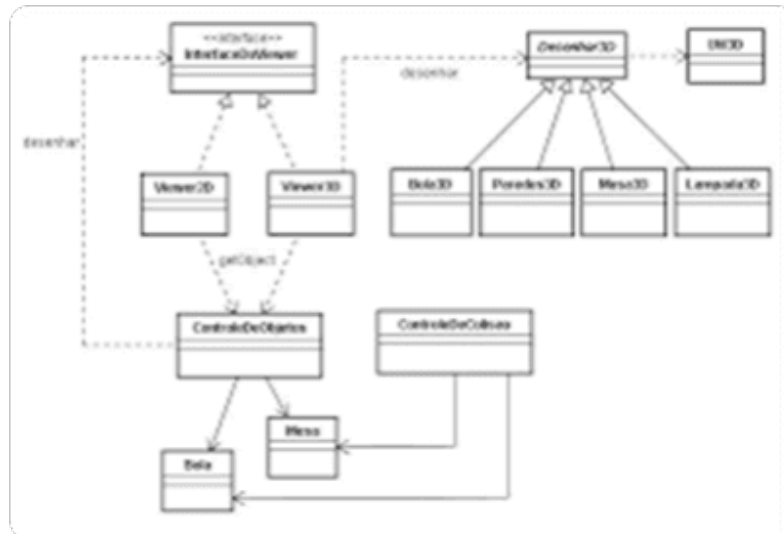
Especificação de interface

A especificação de interfaces de componentes é a forma de descrever como os elementos que quiserem utilizar esses componentes:

- Devem acessá-lo (sua assinatura, com os parâmetros e retornos oferecidos);
- Quais os atributos que ele manipula;
- Quais as funções disponíveis para uso.

Os componentes podem ser agrupados em pacotes, que reúnem componentes com funcionalidades e atributos complementares, dentro do conceito de família de componentes.

Esses pacotes vão apresentar as informações necessárias para que possam ser utilizados no conceito de arquitetura de componentes, através de suas funcionalidades e operações.



Atividade

Os modelos conceituais utilizados na UML garantem que os elementos de negócio, referentes ao sistema que está sendo desenvolvido, sejam completamente mapeados. Garantem também que haja uma migração consistente dos modelos conceituais de negócio para os modelos conceituais de sistemas.

Dessa maneira, conseguimos que nossos sistemas atendam a seus objetivos e sejam desenvolvidos, segundo o melhor conjunto de boas práticas.



Você acha que podemos fazer um paralelo entre esse conceito e a construção de um equipamento, atendendo às normas da ABNT e definições do INMETRO?

Chave de resposta

O paralelo proposto com a construção de um equipamento conforme as normas da ABNT e as diretrizes do INMETRO é pertinente e enriquecedor. Ambos os processos — o desenvolvimento de sistemas baseados em modelos e a fabricação de produtos segundo normas técnicas — compartilham o mesmo objetivo: garantir qualidade, segurança, padronização e conformidade. Assim como as normas técnicas estabelecem requisitos e critérios claros para a produção de equipamentos, os modelos conceituais estruturam o desenvolvimento de software de forma sistemática e rastreável.

Essa analogia é didaticamente útil para a compreensão da importância dos modelos e padrões na engenharia, seja ela de software ou de produtos físicos. Recomenda-se aprofundar essa comparação com exemplos concretos, o que pode enriquecer ainda mais a análise crítica e demonstrar maior domínio conceitual sobre o tema.

Verificando o aprendizado

Questão 1

UML é uma linguagem de modelagem de uso geral que pode ser usada com os principais métodos de objetos e de componentes, podendo ainda ser aplicada a todos os domínios de aplicação e plataformas de implementação;

Um relacionamento estendido entre casos de uso significa que o caso de uso base incorpora implicitamente, sob alguma condição, o comportamento de outro caso de uso.

Um relacionamento de inclusão entre casos de uso significa que o caso de uso base incorpora explicitamente o comportamento de outro caso de uso.

Relacionamentos de inclusão e extensão são representados pela mesma notação do relacionamento de dependência, com a seta apontada para o caso de uso base.

A alternativa completamente CORRETA é:

A I – II – III

B I – III – IV

C II – III – IV

D I – III – IV

E I – II – III -IV



A alternativa A está correta.

As afirmações I – II e III estão alinhadas com os conceitos da UML sobre casos de uso.

Questão 2

Considerando as sentenças a seguir, marque a opção correta:

I. Diagramas de casos de uso são compostos por atores, casos de uso e seus relacionamentos;

II. Diagramas de classes são compostos por classes e seus relacionamentos;

III. Diagramas de sequência são compostos por objetos e suas trocas de mensagens, ou seja, chamadas de métodos para outros objetos;

Considerando as sentenças acima, marque a opção correta:

☐ A Somente as afirmativas I e III são verdadeiras.

☐ B Somente as afirmativas II e III são verdadeiras.

☐ C Somente as afirmativas I e II são verdadeiras.

☒ D As afirmativas I, II e III são verdadeiras.

☐ E Somente a alternativa I é verdadeira.



A alternativa D está correta.

As sentenças I, II e III descrevem com precisão os respectivos diagramas da UML.

Questão 3

O diagrama UML, que apresenta um conjunto de objetos envolvidos em um cenário e a especificação das mensagens trocadas entre esses objetos ao longo de uma linha de tempo, é chamado de:

☐ A Diagrama de mensagens.

☒ B Diagrama de sequência.

☐ C Diagrama de execução em linha de tempo.

☐ D Diagrama de estado.

☐ E Nenhuma das respostas anteriores



A alternativa B está correta.

Esse tipo de diagrama representa a interação entre objetos ao longo do tempo.

Questão 4

Em UML, são exemplos de diagramas responsáveis por representar o aspecto estrutural de um software:

☒ A Diagrama de classes e diagrama de pacotes.

☐ B Diagrama de entidade relacionamento e diagrama de pacotes.

☐ C Diagramas de classes, diagrama de pacotes e use case.

☐ D Use case e diagrama de classes.

☐ E Nenhuma das respostas anteriores.



A alternativa A está correta.

Esses diagramas mostram a estrutura estática do sistema em UML.

Questão 5

A atividade de análise de requisitos procura descobrir o que os stakeholders de um projeto de sistema de software querem que o sistema faça. Para ajudar na comunicação com os usuários e clientes vários diagramas da UML podem ser utilizados.

Com relação à utilização dos diagramas da UML na atividade de análise de requisitos, analise se as sentenças a seguir são falsas ou verdadeiras e depois assinale a opção correta:

() Diagrama de classes desenhado a partir da perspectiva conceitual é uma boa maneira de construir um vocabulário rigoroso do domínio.

() Um diagrama de atividades é recomendado para exibir o fluxo de trabalho da organização, mostrando como o software e as atividades humanas interagem.

() Um diagrama de objetos é indicado para representar um conceito que tenha um ciclo de vida com vários estados e os eventos que mudam esses estados.

A F, V e F

B F, V e V

C V, F e F

D V, V e F

E F, F e V



A alternativa D está correta.

Apenas a sentença: Um diagrama de objetos é indicado para representar um conceito que tenha um ciclo de vida com vários estados e os eventos que mudam esses estados, está incorreta quanto ao uso adequado do diagrama de objetos.

Questão 6

A UML é uma famosa linguagem usada para análise e projeto orientado a objetos. O seu diagrama de sequência tem como características, EXCETO:

A Geralmente é construído a partir dos Casos de Uso.

B Usado para representar interações entre objetos de um cenário, realizadas através de operações ou métodos.

C Dá ênfase ao fluxo de controle de uma atividade para outra.

D Dá ênfase à ordenação temporal em que as mensagens são trocadas entre os objetos de um sistema.

E Pode ser facilmente transformado em um Diagrama de Colaboração.



A alternativa C está correta.

Essa característica é do diagrama de atividades, não do diagrama de sequência.

Considerações finais

Nesta seção, refletimos sobre como enfrentar a complexidade no desenvolvimento de sistemas por meio da estratégia de “dividir para conquistar”. Ao fragmentar grandes problemas em componentes menores — como objetos de negócio, dados e interfaces — torna-se possível desenvolver e integrar esses módulos de forma mais eficiente, resultando em soluções robustas e funcionais. Essa abordagem modular exige organização, e é nesse ponto que as metodologias se tornam fundamentais.

Discutimos a importância das metodologias como guias estruturados que reúnem boas práticas e processos consolidados, permitindo o avanço dos projetos mesmo com equipes em formação ou com pouca experiência. Ao aplicar metodologias tanto de gestão quanto de desenvolvimento, é possível obter maior maturidade na condução dos trabalhos e assegurar entregas mais alinhadas às necessidades do negócio.

Também abordamos como essas metodologias se desdobram em etapas progressivas — desde a modelagem conceitual até a implementação dos sistemas — e como cada etapa contribui para o sucesso do produto final. Nesse contexto, destacamos a Unified Modeling Language (UML) como ferramenta essencial para representar graficamente a arquitetura, o comportamento e as interações do sistema. A UML facilita a comunicação entre os envolvidos no projeto, promovendo clareza e padronização nas decisões.

Em síntese, compreender e aplicar metodologias, aliadas ao uso de modelos visuais como os da UML, é essencial para desenvolver sistemas eficazes e eficientes, com maior controle, previsibilidade e qualidade em cada fase do processo.

Explore +

Para saber mais sobre os assuntos estudados nesta aula, sugerimos:

- Acesse o site da School of Computer Science e busque pelo termo “Software Architecture”.
- Leia o artigo “[Arquitetura de sistemas de Informação - Evolução e Análise Comparativa de Modelos](#)”.
- Acesse o site DEVMEDIA e veja a “Metodologia de desenvolvimento de software”.
- Acesse o site oficial da UML.

Referências

- BASS, Len; CLEMENTS, Paul. **Software Architecture in Practice**. 2. ed. Nova Jersey: Pearson, 2003.
- CHESSMAN, John Daniels. **UML Components** – A Simple Process For Specifying Component -Based Software. Boston: Addison-Wesley, 2000.

- SILVEIRA, Guilherme. **Introdução à arquitetura de design de software**. Rio de Janeiro: Elsevier (Livros Digitais), 2015.
- TERUEL, Evandro Carlos. **Arquitetura de sistemas para Web com Java Utilizando Design Patterns e Frameworks**. Rio de Janeiro: Ciência Moderna, 2012.